

Informe de Laboratorio 04

Tema: Arbol Binario de Búsqueda

Nota

Estudiante	Escuela	Asignatura
Sebastian Arley Chirinos Negrón schirinosne@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Estructura de datos y Algoritmos Semestre: I Código: 1702124

Laboratorio	Tema	Duración
04	Arbol Binario de Búsqueda	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	Del 12 Junio 2023	Al 19 Junio 2023

1. Tarea

- Implementación de Árboles Binarios de Búsqueda:
- search(): Esta operación busca un elemento específico en el árbol binario de búsqueda. Comienza en la raíz y realiza comparaciones para determinar si el elemento buscado es menor, igual o mayor que el valor del nodo actual. Luego, se desplaza hacia el subárbol izquierdo o derecho según corresponda hasta encontrar el elemento o llegar a un nodo nulo.
- getMin(): Esta operación devuelve el valor mínimo almacenado en el árbol binario de búsqueda. Se realiza recorriendo siempre por los nodos más a la izquierda hasta llegar al nodo hoja más izquierdo.
- getMax(): Esta operación devuelve el valor máximo almacenado en el árbol binario de búsqueda. Se realiza recorriendo siempre por los nodos más a la derecha hasta llegar al nodo hoja más derecho.
- parent(): Esta operación devuelve el valor del nodo padre de un elemento dado en el árbol binario de búsqueda. Se realiza buscando el elemento y almacenando una referencia al nodo padre durante el recorrido.
- son(): Esta operación verifica si un elemento dado tiene hijos en el árbol binario de búsqueda. Se realiza buscando el elemento y verificando si tiene nodos hijos izquierdo o derecho.
- insert(): Esta operación permite insertar un nuevo elemento en el árbol binario de búsqueda. Comienza comparando el nuevo elemento con el valor del nodo actual y se desplaza hacia el subárbol izquierdo o derecho según corresponda hasta encontrar una posición vacía para la inserción.

- `remove()`: Esta operación permite eliminar un elemento específico del árbol binario de búsqueda. Se realiza buscando el elemento y aplicando diferentes casos dependiendo de si el nodo a eliminar es una hoja, tiene un solo hijo o tiene dos hijos.
- **INPUT**: Una sola palabra en mayúsculas.
- **OUTPUT**: Se debe construir el árbol binario de búsqueda considerando el valor decimal de su código ASCII.
- **Pruebas de las operaciones implementadas**: Para verificar el correcto funcionamiento de las operaciones, se realizarán pruebas utilizando diferentes palabras y se mostrarán los resultados obtenidos.
- **Estudio de la librería Graph Stream**: Se investigará la librería Graph Stream para obtener una salida gráfica de la implementación del árbol binario de búsqueda. Se explorarán las funcionalidades y se evaluará su viabilidad para representar visualmente la estructura y los nodos del árbol.
- Se aplicarán todas las recomendaciones dadas por el docente para asegurar una implementación eficiente y correcta del árbol binario de búsqueda.

2. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- <https://github.com/BastleyNait/EDA-LAB-B-23A.git>
- URL para el laboratorio 04 en el Repositorio GitHub.
- <https://github.com/BastleyNait/EDA-LAB-B-23A/tree/main/lab03>

3. Ejercicios designados

3.1. Clase Bst

- Un BST (Binary Search Tree, por sus siglas en inglés) es una estructura de datos jerárquica utilizada para almacenar elementos de manera ordenada y permitir búsquedas eficientes. Consiste en un árbol binario en el que cada nodo tiene un valor asociado y dos hijos, uno izquierdo y otro derecho. La propiedad principal de un BST es que el valor de cada nodo en el subárbol izquierdo es menor que el valor del nodo y el valor de cada nodo en el subárbol derecho es mayor. Esto facilita la búsqueda de un elemento específico en el árbol, ya que se puede comparar el valor buscado con los valores de los nodos y seguir el camino apropiado hacia la izquierda o la derecha según corresponda. Además de las operaciones de búsqueda, los BST también admiten inserción, eliminación y recorridos en orden, preorden y postorden.

Listing 1: Bst.java

```
1 import myExceptions.ExceptionNotFound;
2
3 public class Bst<T extends Comparable<T>> {
4     private Nodo <T> root;
5
6     public Bst() {
7         this.root = null;
```

```
8      }
9
10     public Nodo <T> getRoot() {
11         return root;
12     }
13
14     public void setRoot(Nodo <T> root) {
15         this.root = root;
16     }
17
18     public boolean isEmpty() {
19         return this.root == null;
20     }
21
22     public void insert(T x) throws ExceptionNotFound {
23         this.root = insert(x, this.root);
24     }
25
26     private Nodo <T> insert(T x, Nodo <T> current) throws ExceptionNotFound {
27         Nodo <T> hoja = current;
28         if (current == null) {
29             hoja = new Nodo <>(x);
30         } else {
31             int bool = current.getData().compareTo(x);
32             if (bool == 0)
33                 throw new ExceptionNotFound("Ya existe el elemento");
34             if (bool < 0)
35                 hoja.setRight(insert(x, current.getRight()));
36             else
37                 hoja.setLeft(insert(x, current.getLeft()));
38         }
39         return hoja;
40     }
41
42     public T search(T x) throws ExceptionNotFound {
43         Nodo <T> aux = search(x, this.root);
44         if (aux == null) {
45             throw new ExceptionNotFound("No se encontr el elemento");
46         }
47         return aux.getData();
48     }
49     public Nodo <T> searchNode(T x) throws ExceptionNotFound {
50         Nodo <T> aux = search(x, this.root);
51         if (aux == null) {
52             throw new ExceptionNotFound("No se encontr el elemento");
53         }
54         return aux;
55     }
56
57     private Nodo <T> search(T x, Nodo <T> current) {
58         if (current == null) {
59             return null;
60         } else {
61             int bool = current.getData().compareTo(x);
62             if (bool == 0)
63                 return current;
```

```
64         if (bool < 0)
65             return search(x, current.getRight());
66         else
67             return search(x, current.getLeft());
68     }
69 }
70
71 private T getMin(Nodo <T> node) {
72     T minKey = node.getData();
73     while (node.getLeft() != null) {
74         minKey = node.getLeft().getData();
75         node = node.getLeft();
76     }
77     return minKey;
78 }
79
80 public T getMin() throws ExceptionNotFound {
81     if (isEmpty())
82         throw new ExceptionNotFound("el arbol esta vacio");
83     Nodo <T> current = root;
84     while (current.getLeft() != null) {
85         current = current.getLeft();
86     }
87     return current.getData();
88 }
89
90 public T getMax() throws ExceptionNotFound {
91     if (isEmpty())
92         throw new ExceptionNotFound("el arbol esta vacio");
93     Nodo <T> current = root;
94     while (current.getRight() != null) {
95         current = current.getRight();
96     }
97     return current.getData();
98 }
99
100 public T parent(T x) throws ExceptionNotFound {
101     if (findParent(x, this.root) == null)
102         throw new ExceptionNotFound("El nodo no tiene padre");
103     return findParent(x, this.root).getData();
104 }
105
106 private Nodo <T> findParent(T x, Nodo <T> current) {
107     if (current == null || current.getData().equals(x))
108         return null;
109     /*
110      * comparamos si el nodo izq o der del nodo actual es igual al dato que le
111      * pasamos
112      */
113     if (current.getLeft() != null && current.getLeft().getData().equals(x) ||
114         current.getRight() != null && current.getRight().getData().equals(x))
115         return current;
116     /*
117      * Si no es as volvemos a llamar a la funcion hasta que encontremos
118      */
119     int bool = x.compareTo(current.getData());
```

```

119         if (bool < 0)
120             return findParent(x, current.getLeft());
121         else
122             return findParent(x, current.getRight());
123     }
124
125     public void remove(T x) throws ExceptionNotFound {
126         this.root = remove(x, this.root);
127     }
128
129     private Nodo <T> remove(T x, Nodo <T> current) throws ExceptionNotFound {
130         if (current == null)
131             throw new ExceptionNotFound("No se encontr el elemento");
132
133         int bool = current.getData().compareTo(x);
134         if (bool < 0)
135             current.setRight(remove(x, current.getRight()));
136         else if (bool > 0)
137             current.setLeft(remove(x, current.getLeft()));
138         else {
139             if (current.getLeft() == null)
140                 return current.getRight();
141             else if (current.getRight() == null)
142                 return current.getLeft();
143
144             current.setData(getMin(current.getRight()));
145             current.setRight(remove(current.getData(), current.getRight()));
146         }
147
148         return current;
149     }
150
151     public void inOrden() {
152         if (isEmpty())
153             System.out.println("arbol vaceo papay");
154         else
155             inOrden(this.root);
156     }
157
158     public void inOrden(Nodo <T> current) {
159         if (current.getLeft() != null)
160             inOrden(current.getLeft());
161         System.out.println(current);
162         if (current.getRight() != null)
163             inOrden(current.getRight());
164     }
165 }

```

- **Line 1:** Importa la clase ExceptionNotFound del paquete myExceptions.
- **Line 3:** Declaración de la clase Bst que utiliza un tipo genérico T que extiende la interfaz Comparable.
- **Line 4:** Declaración del atributo root de tipo Nodo<T>, que representa la raíz del árbol binario de búsqueda.

- **Line 6-9:** Constructor de la clase Bst que inicializa el atributo root como nulo.
- **Line 11-14:** Método getter para obtener el valor del atributo root.
- **Line 16-19:** Método setter para establecer el valor del atributo root.
- **Line 21-24:** Método que verifica si el árbol binario de búsqueda está vacío. Retorna true si el atributo root es nulo.
- **Line 26-36:** Método insert() que permite insertar un nuevo elemento en el árbol binario de búsqueda. Recibe el elemento a insertar y el nodo actual. Compara el elemento con el valor del nodo actual y se desplaza hacia el subárbol izquierdo o derecho según corresponda hasta encontrar una posición vacía para la inserción. Si el elemento ya existe en el árbol, lanza una excepción de tipo ExceptionNotFound.
- **Line 38-61:** Método search() que busca un elemento específico en el árbol binario de búsqueda. Recibe el elemento a buscar y el nodo actual. Realiza comparaciones para determinar si el elemento buscado es menor, igual o mayor que el valor del nodo actual y se desplaza hacia el subárbol izquierdo o derecho según corresponda. Retorna el nodo encontrado. Si el elemento no se encuentra en el árbol, lanza una excepción de tipo ExceptionNotFound.
- **Line 63-76:** Método getMin() que devuelve el valor mínimo almacenado en el árbol binario de búsqueda. Recibe un nodo y se desplaza siempre por los nodos más a la izquierda hasta llegar al nodo hoja más izquierdo. Retorna el valor mínimo encontrado. Si el árbol está vacío, lanza una excepción de tipo ExceptionNotFound.
- **Line 78-91:** Método getMax() que devuelve el valor máximo almacenado en el árbol binario de búsqueda. Recibe un nodo y se desplaza siempre por los nodos más a la derecha hasta llegar al nodo hoja más derecho. Retorna el valor máximo encontrado. Si el árbol está vacío, lanza una excepción de tipo ExceptionNotFound.
- **Line 93-102:** Método parent() que devuelve el valor del nodo padre de un elemento dado en el árbol binario de búsqueda. Recibe el elemento a buscar y el nodo actual. Busca el nodo padre durante el recorrido y retorna su valor. Si el nodo no tiene padre, lanza una excepción de tipo ExceptionNotFound.
- **Line 104-139:** Método remove() que permite eliminar un elemento específico del árbol

3.2. Clase Nodo

- El código proporcionado define la clase Nodo que se utiliza en la implementación de un árbol binario de búsqueda. Los nodos contienen un valor y referencias a los nodos hijos izquierdo y derecho. También se incluyen métodos getter y setter para acceder y modificar los atributos del nodo, así como un método toString() para representar el valor del nodo como una cadena.

Listing 2: Nodo.java

```
1 public class Nodo <T> {  
2     private T data;  
3  
4     private Nodo <T> left;  
5  
6     private Nodo <T> right;  
7  
8     public Nodo(T data) {
```

```
9      this(data,null,null);
10    }
11    public Nodo(T data, Nodo <T> left, Nodo <T> right) {
12        this.data = data;
13        this.left = left;
14        this.right = right;
15    }
16    public T getData() {
17        return this.data;
18    }
19
20    public void setData(T data) {
21        this.data = data;
22    }
23    public Nodo <T> getLeft() {
24        return this.left;
25    }
26
27    public void setLeft(Nodo <T> left) {
28        this.left = left;
29    }
30
31    public Nodo <T> getRight() {
32        return this.right;
33    }
34
35    public void setRight(Nodo <T> right) {
36        this.right = right;
37    }
38
39    public String toString() {
40        return this.data.toString();
41    }
42
43    //recorridos en profundity
44    /*
45     * Raiz
46     * Izquierda
47     * Derecha
48     * */
49    // pre-order Rid: 3 1 6 4 5 8 7 9
50    // in-order iRd: 1 3 5 4 6 7 8 9
51    // post-order idR: 1 5 4 7 9 8 6 3
52
53    // busqueda binary/dichotomy
54
55 }
```

- **Line 1:** Declaración de la clase Nodo con un parámetro de tipo genérico T.
- **Line 3:** Declaración del atributo data de tipo T, que almacena el valor del nodo.
- **Line 5-7:** Declaración de los atributos left y right de tipo Nodo<T>, que representan los hijos izquierdo y derecho del nodo respectivamente.
- **Line 9-11:** Constructor de la clase Nodo que recibe el valor del nodo, el nodo hijo izquierdo y el nodo hijo derecho. Inicializa los atributos correspondientes con los valores proporcionados.

- **Line 13-16:** Constructor sobrecargado de la clase Nodo que recibe solo el valor del nodo. Llama al constructor principal con los valores proporcionados y establece los hijos izquierdo y derecho como nulos.
- **Line 18-21:** Método getter para obtener el valor del atributo data.
- **Line 23-26:** Método setter para establecer el valor del atributo data.
- **Line 28-31:** Método getter para obtener el nodo hijo izquierdo.
- **Line 33-36:** Método setter para establecer el nodo hijo izquierdo.
- **Line 38-41:** Método getter para obtener el nodo hijo derecho.
- **Line 43-46:** Método setter para establecer el nodo hijo derecho.
- **Line 48-51:** Método toString() que devuelve una representación en cadena del valor del nodo.

3.3. Clase Aplicacion

- El código proporcionado define la clase Nodo que se utiliza en la implementación de un árbol binario de búsqueda. Los nodos contienen un valor y referencias a los nodos hijos izquierdo y derecho. También se incluyen métodos getter y setter para acceder y modificar los atributos del nodo, así como un método toString() para representar el valor del nodo como una cadena.

Listing 3: Aplicacion.java

```
1 import myExceptions.ExceptionNotFound;
2 import org.graphstream.graph.Graph;
3 import org.graphstream.graph.implementations.SingleGraph;
4 import org.graphstream.graph.Node;
5 import org.graphstream.graph.Edge;
6 import org.graphstream.ui.view.Viewer;
7
8
9 import java.util.Iterator;
10 import java.util.Scanner;
11
12 public class Aplicacion {
13     public static void main(String args[]) throws ExceptionNotFound {
14         generar();
15     }
16
17     public static void generar() throws ExceptionNotFound {
18         System.setProperty("org.graphstream.ui", "swing");
19
20         Graph graph = new SingleGraph("Aplicacion");
21
22         Viewer viewer = graph.display();
23         viewer.disableAutoLayout();
24
25         graph.setAttribute("ui.stylesheet", styleSheet);
26         Scanner sc = new Scanner(System.in);
27         System.out.println("ingrese palabra en mayusculas :");
28         String word = sc.next(); // Ingresa tu palabra en mayusculas aqu
29
30         Bst<String> bst = new Bst<>();
```

```
31     int i = 0;
32
33     char[] words = word.toCharArray();
34     for (char c : words) {
35         try {
36             try {
37                 bst.search(String.valueOf(c));
38                 // Si la bsqueda no arroja una excepcin, se ejecuta este bloque
39                 bst.insert(String.valueOf(c) + "-" + i);
40
41             } catch (ExceptionNotFound e) {
42                 bst.insert((String.valueOf(c)));
43             }
44
45         } catch (ExceptionNotFound e) {
46             e.printStackTrace();
47         }
48     }
49     addNodesToGraph(bst.getRoot(), graph, 0.0, 0.0);
50     Graph graph2 = new SingleGraph("Aplicacion2");
51     graph2.setAttribute("ui.stylesheet", styleSheet);
52     Viewer viewer2 = graph2.display();
53     viewer2.disableAutoLayout();
54
55     System.out.println("ingrese letra a eliminar :");
56     String eliminar = sc.next(); // Ingresa tu palabra en mayusculas aqu
57     bst.remove(eliminar);
58     graph.removeNode(eliminar);
59     addNodesToGraph(bst.getRoot(), graph2, 0.0, 0.0);
60
61     System.out.println("Minimo valor: " + bst.getMin());
62     Node node1 = graph2.addNode(String.valueOf(bst.getMin()) + "-");
63     node1.setAttribute("ui.label", node1.getId());
64     node1.setAttribute("xy", 2, 2);
65
66     System.out.println("Maximo valor: " + bst.getMax());
67     Node node2 = graph2.addNode(String.valueOf(bst.getMax()) + "+");
68     node2.setAttribute("ui.label", node2.getId());
69     node2.setAttribute("xy", 3, 2);
70
71     System.out.println("ingrese letra a buscar:");
72     String buscar = sc.next(); // Ingresa tu palabra en mayusculas aqu
73     System.out.println("buscar " + buscar + ": " + bst.search(buscar));
74     System.out.println("parent " + buscar + ": " + bst.parent(buscar));
75     Node node3 = graph2.addNode(String.valueOf(bst.search(buscar)) + "-s");
76     Node node4 = graph2.addNode(String.valueOf(bst.parent(buscar)) + "-p");
77     node3.setAttribute("ui.label", node3.getId());
78     node3.setAttribute("xy", 4, 2);
79
80     node4.setAttribute("ui.label", node4.getId());
81     node4.setAttribute("xy", 5, 2);
82
83     System.out.println("ingrese letra a nodo a buscar sons: ");
84     String son = sc.next(); // Ingresa tu palabra en mayusculas aqu
85 }
86
```

```

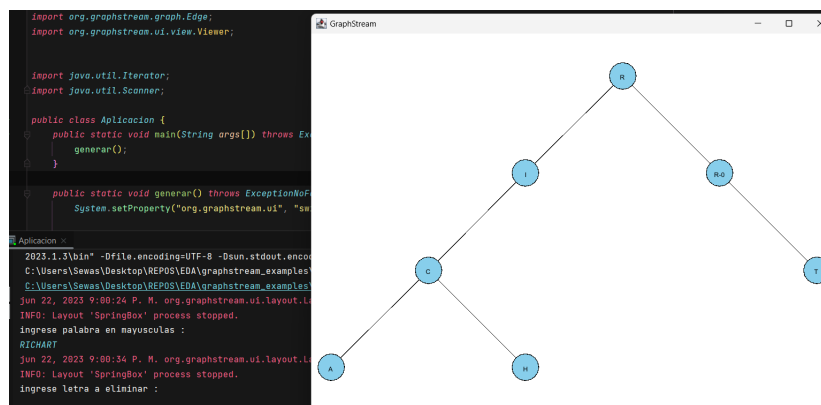
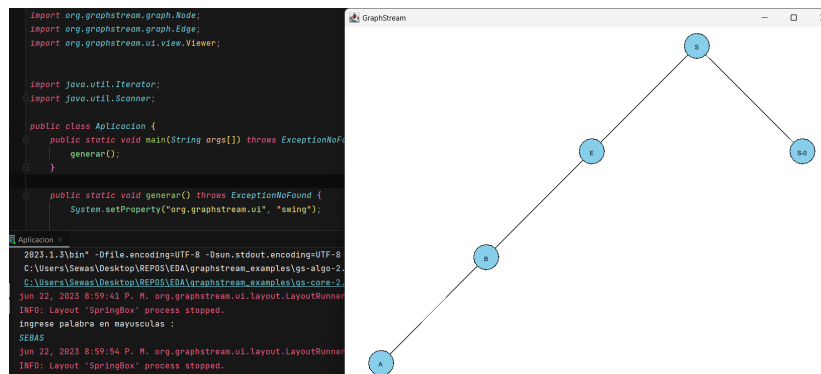
87     private static void addNodesToGraph(Nodo current, Graph graph, double x, double y) {
88         if (current != null) {
89             // Agregar el nodo actual al grafo
90             Node node = graph.addNode(String.valueOf(current.getData()));
91             node.setAttribute("ui.label", node.getId());
92             node.setAttribute("xy", x, y);
93
94             double leftX = x - 1.0;
95             double rightX = x + 1.0;
96             double childY = y - 1.0;
97
98             // Agregar los nodos hijos recursivamente
99             if (current.getLeft() != null) {
100                 addNodesToGraph(current.getLeft(), graph, leftX, childY);
101                 graph.addEdge((current.getData() + "-" + current.getLeft().getData()),
102                     String.valueOf(current.getData()),
103                     String.valueOf(current.getLeft().getData()));
104             }
105             if (current.getRight() != null) {
106                 addNodesToGraph(current.getRight(), graph, rightX, childY);
107                 graph.addEdge((current.getData() + "-" + current.getRight().getData()),
108                     String.valueOf(current.getData()),
109                     String.valueOf(current.getRight().getData()));
110             }
111         }
112     }
113
114     protected static String styleSheet = "node {" +
115         "    shape: circle;" +
116         "    size: 40px;" +
117         "    text-size: 12;" +
118         "    fill-mode: plain;" +
119         "    fill-color: skyblue;" +
120         "    stroke-mode: plain;" +
121         "    stroke-color: black;" +
122         "    stroke-width: 1px;" +
123         "}" +
124         "edge {" +
125         "    arrow-shape: arrow;" +
126         "    arrow-size: 20px, 4px;" +
127         "}" +
128         "node.root {" +
129         "    fill-color: yellow;" +
130         "}";

```

- **Line 10:** Método principal main que se ejecuta al iniciar la aplicación. Llama al método generar().
- **Line 13-15:** Método generar() que genera el árbol binario de búsqueda, crea y muestra el grafo utilizando la biblioteca GraphStream, y realiza diversas operaciones en el árbol.
- **Line 17:** Establece la propiedad del sistema `.org.graphstream.ui.a "swing"`. Esta propiedad configura el renderizador de GraphStream para utilizar la interfaz de usuario de Swing.
- **Line 19-20:** Crea un objeto Graph llamado graph utilizando la implementación Single-Graph. Este objeto representa el grafo que se visualizará.

- **Line 22-23:** Crea un objeto Viewer y lo asocia al grafo graph. El Viewer se utiliza para visualizar el grafo en una ventana.
- **Line 24:** Deshabilita el diseño automático del grafo. Esto evita que los nodos se reorganicen automáticamente.
- **Line 26-27:** Establece una hoja de estilo para el grafo. Define cómo se mostrarán los nodos y las aristas en la visualización.
- **Line 28-30:** Crea un objeto Scanner para leer la entrada del usuario.
- **Line 31-33:** Solicita al usuario que ingrese una palabra en mayúsculas y la guarda en la variable word.
- **Line 35-37:** Crea un objeto Bst<String> llamado bst utilizando la implementación de un árbol binario de búsqueda. Este árbol se utilizará para almacenar los caracteres de la palabra.
- **Line 38:** Inicializa la variable i con el valor 0.
- **Line 40-42:** Convierte la palabra en un arreglo de caracteres.
- **Line 43-66:** Itera sobre cada carácter en el arreglo de caracteres.
- **Line 45-61:** Dentro del bucle, se realiza lo siguiente:
 - Se intenta buscar el carácter en el árbol utilizando el método search(). Si la búsqueda es exitosa, se lanza una excepción ExceptionNotFound, que indica que el elemento ya existe en el árbol.
 - Si no se produce una excepción, se inserta el carácter junto con un sufijo numérico en el árbol utilizando el método insert().
 - Si se produce una excepción ExceptionNotFound, se captura y se imprime la traza de la excepción.
- **Line 68-69:** Agrega los nodos y aristas al grafo graph utilizando el método addNodesToGraph(). El método recorre el árbol y agrega los nodos correspondientes al grafo, así como las aristas que los conectan.
- **Line 70-71:** Crea un nuevo objeto Graph llamado graph2 utilizando la implementación SingleGraph. Este objeto representa el segundo grafo en el que se mostrará el árbol después de realizar la eliminación de un nodo.
- **Line 73:** Establece la hoja de estilo para el grafo graph2 de manera similar a como se hizo para el grafo graph.
- **Line 74-76:** Crea un objeto Viewer asociado al grafo graph2, que se utilizará para visualizar el grafo en una ventana.
- **Line 77:** Deshabilita el diseño automático del grafo graph2.
- **Line 79-80:** Solicita al usuario que ingrese una letra a eliminar y la guarda en la variable eliminar.
- **Line 81-82:** Llama al método remove(eliminar) del árbol binario de búsqueda bst para eliminar el nodo correspondiente a la letra ingresada.
- **Line 83:** Remueve el nodo correspondiente a la letra ingresada del grafo graph.
- **Line 84-85:** Agrega los nodos y aristas actualizados al grafo graph2 utilizando nuevamente el método addNodesToGraph().
- **Line 87-88:** Imprime el mínimo valor del árbol binario de búsqueda utilizando el método getMin().
- **Line 89-90:** Crea un nuevo nodo en el grafo graph2 representando el mínimo valor del árbol y lo posiciona en la visualización.

- **Line 92-93:** Imprime el máximo valor del árbol binario de búsqueda utilizando el método getMax().
- **Line 94-95:** Crea un nuevo nodo en el grafo graph2 representando el máximo valor del árbol y lo posiciona en la visualización.
- **Line 97-98:** Solicita al usuario que ingrese una letra a buscar y la guarda en la variable buscar.
- **Line 99-100:** Realiza una búsqueda en el árbol binario de búsqueda utilizando el método search(buscar).
- **Line 101-102:** Realiza una búsqueda del padre de la letra ingresada utilizando el método parent(buscar).
- **Line 103-104:** Crea un nuevo nodo en el grafo graph2 representando el resultado de la búsqueda y lo posiciona en la visualización.
- **Line 105-106:** Crea un nuevo nodo en el grafo graph2 representando el resultado de la búsqueda del padre y lo posiciona en la visualización.
- **Line 108-109:** Solicita al usuario que ingrese una letra para buscar sus hijos y la guarda en la variable son.
- Ejecucion del codigo:



3.4. Commits del trabajo

- Acá estan algunos de los commits mas importantes en este trabajo:

```
commit a612dd4bca519c5af508643799b1aeb68e2bd75b (HEAD -> main, origin/main, origin/HEAD)
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 20:56:34 2023 -0500

    mejoras 1

commit fb6589e4d9bbf4af7368831ac458a6b7c52b68ae
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 20:54:58 2023 -0500

    mejoras

commit 56a25050cd3afebd880b8ef11b589d50042bbb64
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Wed Jun 21 21:14:49 2023 -0500

    implementando un nuevo arbol

commit bd88c4a334723a15cdf4e2ddc2fe3abe3e35f91d
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Tue Jun 20 21:12:57 2023 -0500

    dias acaso me has avandonando?

commit 43d7b09b451363150a2ef71f44a9ba121d24ad04
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Tue Jun 20 13:06:17 2023 -0500

    comentarios
    ...skipping...
commit a612dd4bca519c5af508643799b1aeb68e2bd75b (HEAD -> main, origin/main, origin/HEAD)
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 20:56:34 2023 -0500

    mejoras 1

commit fb6589e4d9bbf4af7368831ac458a6b7c52b68ae
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 20:54:58 2023 -0500

    mejoras

commit 56a25050cd3afebd880b8ef11b589d50042bbb64
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Wed Jun 21 21:14:49 2023 -0500

    implementando un nuevo arbol

commit bd88c4a334723a15cdf4e2ddc2fe3abe3e35f91d
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Tue Jun 20 21:12:57 2023 -0500

    dias acaso me has avandonando?

commit 43d7b09b451363150a2ef71f44a9ba121d24ad04
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Tue Jun 20 13:06:17 2023 -0500

    comentarios
    ...skipping...
commit a612dd4bca519c5af508643799b1aeb68e2bd75b (HEAD -> main, origin/main, origin/HEAD)
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 20:56:34 2023 -0500

    mejoras 1

commit fb6589e4d9bbf4af7368831ac458a6b7c52b68ae
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 20:54:58 2023 -0500

    mejoras

commit 56a25050cd3afebd880b8ef11b589d50042bbb64
Author: BastleyWalt <schirinosne@unsa.edu.pe>
Date: Wed Jun 21 21:14:49 2023 -0500

    implementando un nuevo arbol
```

3.5. Estructura de laboratorio 04

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab04/
| .gitignore
| Bst.java
| Nodo.java
| Aplicacion.java

\---latex
| EDA_lab04_schirinosne.pdf
| EDA_lab04_schirinosne.tex
|
+---img
| Commits.png
| ejecucion1.png
| ejecucion2.png
| logo_abet.png
| logo_episunsa.png
| logo_unsa.jpg
|
\---src
| Bst.java
| Nodo.java
| Aplicacion.java
|
\---myExceptions
| ExceptionIsEmpty.java
| ExceptionNotFound.java
```

4. ¿Explique como es el algoritmo que implemento para obtener el arbol binario de busqueda con la libreria Graph Stream? Recuerde que pueden haber operaciones sobre el BST.

- Se utiliza la biblioteca GraphStream para visualizar el árbol binario de búsqueda.
- Se crea un objeto Graph utilizando la implementación SingleGraph para representar el grafo.
- Se crea un objeto Viewer y se asocia al grafo para visualizarlo en una ventana.
- Se deshabilita el diseño automático del grafo para evitar reorganizaciones automáticas de los nodos.
- Se establece una hoja de estilo para definir la apariencia de los nodos y las aristas en la visualización.
- Se lee una palabra en mayúsculas ingresada por el usuario.
- Se crea un árbol binario de búsqueda y se insertan los caracteres de la palabra en el árbol.
- Se agregan los nodos y las aristas correspondientes al grafo para representar el árbol.
- Se muestra el grafo que representa el árbol binario de búsqueda.

5. Rúbricas

5.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

5.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumple con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

Puntos	Nivel			
	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	x	4	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	2	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	2	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	0	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	x	4	
Total		20		18	

6. Referencias

- <https://algorithmtutor.com/Data-Structures/Tree/Binary-Trees/>
- <https://algorithmtutor.com/Data-Structures/Tree/Binary-Search-Trees/>
- https://github.com/rescobedoq/graphstream_examples